

咖啡拿铁

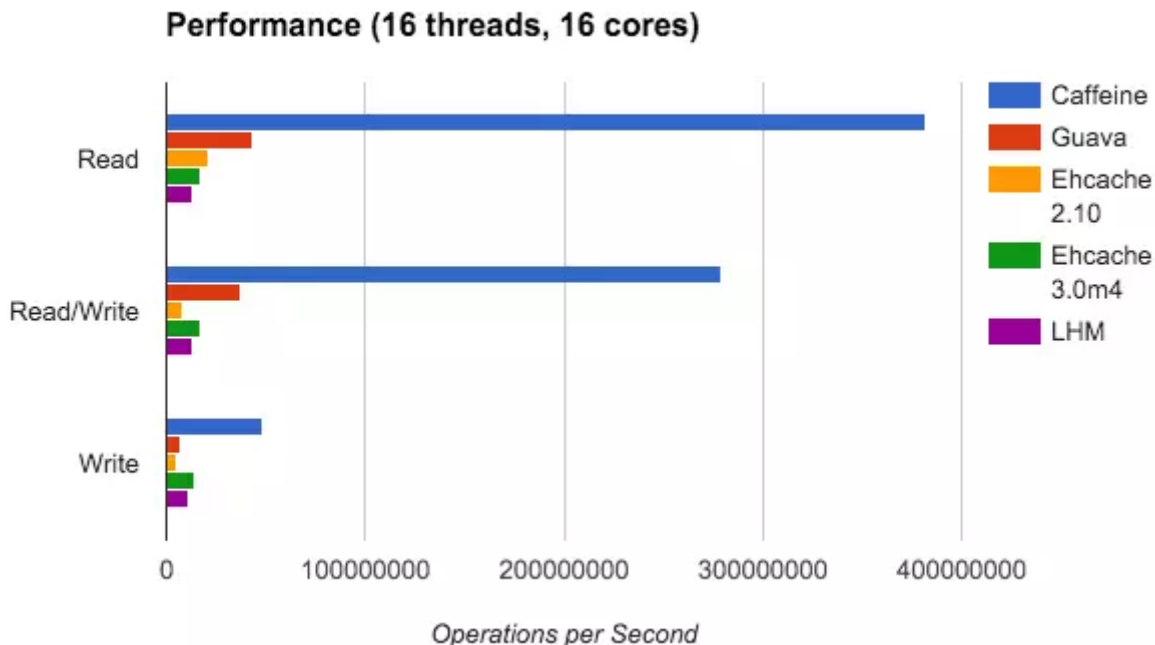
2018年09月05日 阅读 4335

关注

深入解密来自未来的缓存-Caffeine

1.前言

读这篇文章之前希望你能好好的阅读: [你应该知道的缓存进化史](#) 和 [如何优雅的设计和使用缓存?](#)。这两篇文章主要从一些实战上面去介绍如何去使用缓存。在这两篇文章中我都比较推荐Caffeine这款本地缓存去代替你的Guava Cache。本篇文章我将介绍Caffeine缓存的具体有哪些功能, 以及内部的实现原理, 让大家知其然, 也要知其所以然。有人会问:我不使用Caffeine这篇文章应该对我没啥用了, 别着急, 在Caffeine中的知识一定会对你在其他代码设计方面有很大的帮助。当然在介绍之前还是要贴一下他和其他缓存的一些比较图:



可以看见Caffeine基本从各个维度都是相比于其他缓存都高, 废话不多说, 首先还是先看看如何使用吧。

Caffeine使用比较简单，API和Guava Cache一致:

```
public static void main(String[] args) {  
    Cache<String, String> cache = Caffeine.newBuilder()  
        .expireAfterWrite(1, TimeUnit.SECONDS)  
        .expireAfterAccess(1, TimeUnit.SECONDS)  
        .maximumSize(10)  
        .build();  
    cache.put("hello", "hello");  
}
```

2.Caffeine原理简介

2.1W-TinyLFU

传统的LFU受时间周期的影响比较大。所以各种LFU的变种出现了，基于时间周期进行衰减，或者在最近某个时间段内的频率。同样的LFU也会使用额外空间记录每一个数据访问的频率，即使数据没有在缓存中也需要记录，所以需要维护的额外空间很大。

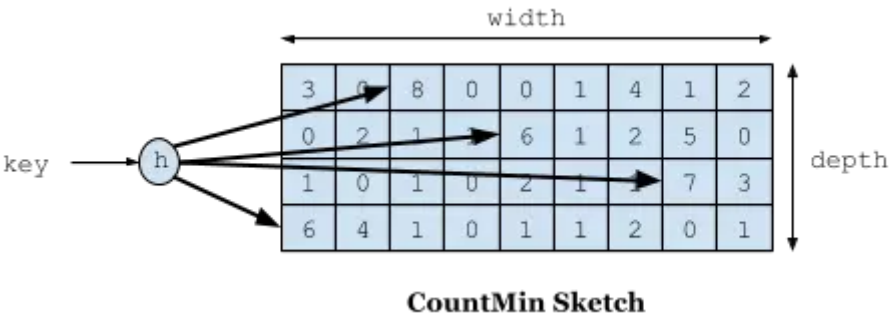
可以试想我们对这个维护空间建立一个hashMap，每个数据项都会存在这个hashMap中，当数据量特别大的时候，这个hashMap也会特别大。

再回到LRU，我们的LRU也不是那么一无是处，LRU可以很好的应对突发流量的情况，因为他不需要累计数据频率。

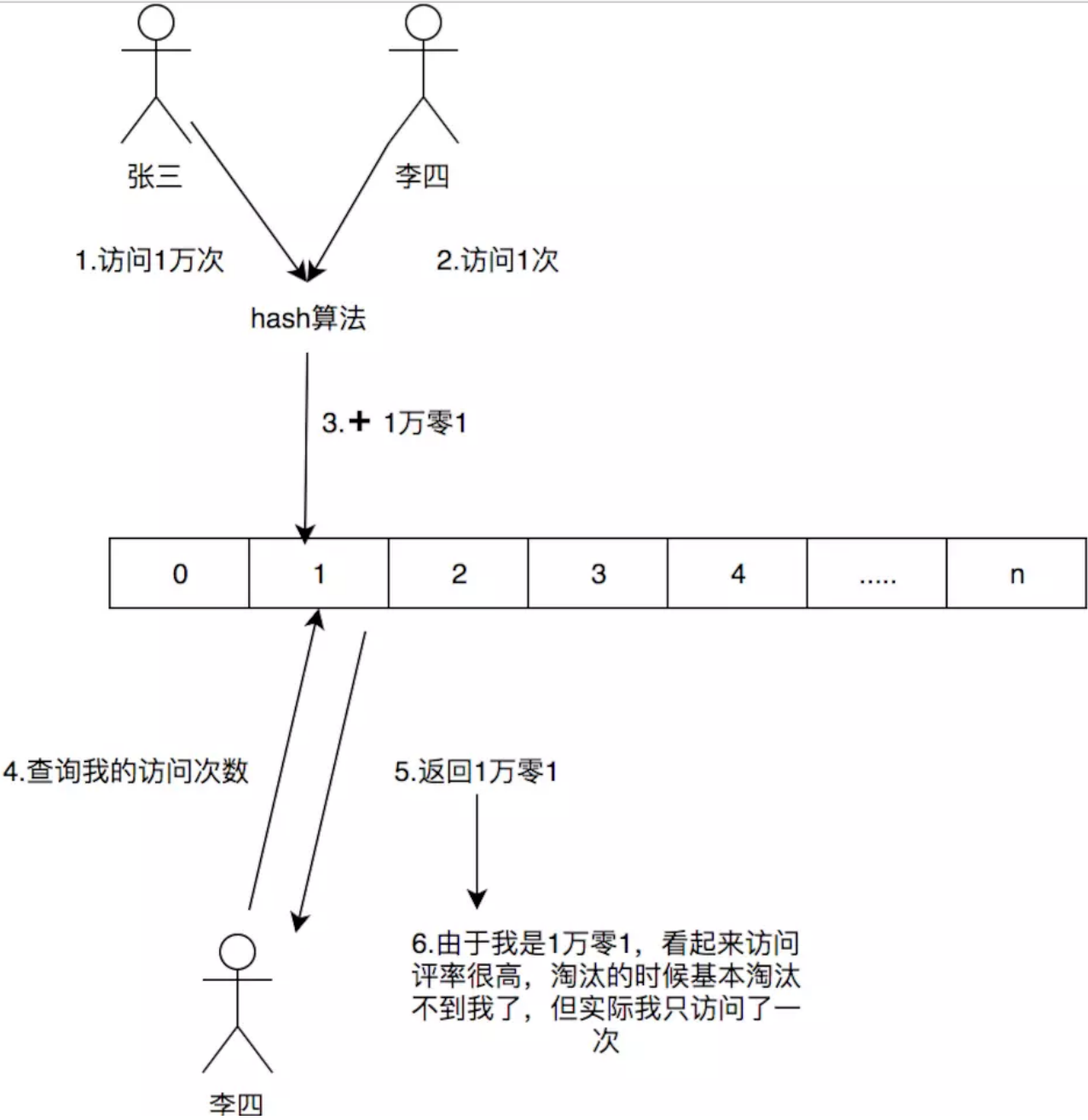
所以W-TinyLFU结合了LRU和LFU，以及其他的算法的一些特点。

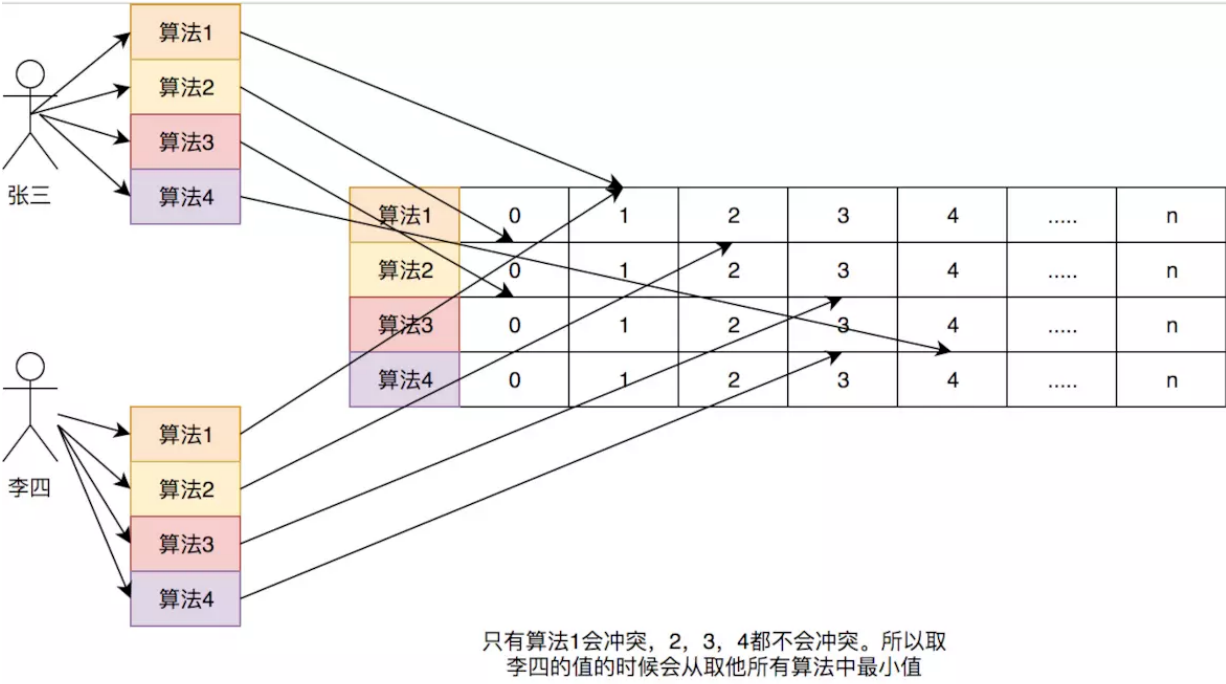
2.1.1频率记录

首先要说到的就是频率记录的问题，我们要实现的目标是利用有限的空间可以记录随时间变化的访问频率。在W-TinyLFU中使用Count-Min Sketch记录我们的访问频率，而这个也是布隆过滤器的一种变种。如下图所示:



如果需要记录一个值，那我们需要通过多种Hash算法对其进行处理hash，然后在对应的hash算法的记录中+1，为什么需要多种hash算法呢？由于这是一个压缩算法必定会出现冲突，比如我们建立一个Long的数组，通过计算出每个数据的hash的位置。比如张三和李四，他们两有可能hash值都是相同，比如都是1那Long[1]这个位置就会增加相应的频率，张三访问1万次，李四访问1次那Long[1]这个位置就是1万零1，如果取李四的访问评率的时候就会取出是1万零1，但是李四命名只访问了1次啊，为了解决这个问题，所以用了多个hash算法可以理解为long[][]二维数组的一个概念，比如在第一个算法张三和李四冲突了，但是在第二个，第三个中很大的概率不冲突，比如一个算法大概有1%的概率冲突，那四个算法一起冲突的概率是1%的四次方。通过这个模式我们取李四的访问率的时候取所有算法中，李四访问最低频率的次数。所以他的名字叫Count-Min Sketch。

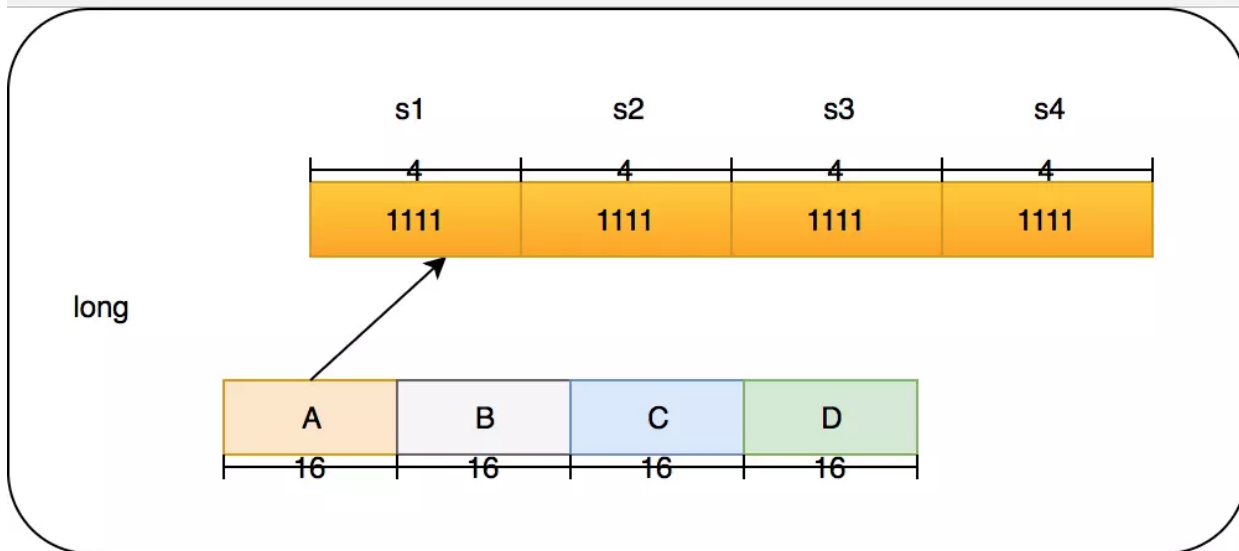




这里和以前的做个对比，简单的举个例子:如果一个hashMap来记录这个频率，如果我有100个数据，那这个HashMap就得存储100个这个数据的访问频率。哪怕我这个缓存的容量是1，因为Lfu的规则我必须全部记录这个100个数据的访问频率。如果有更多的数据我就有记录更多的。

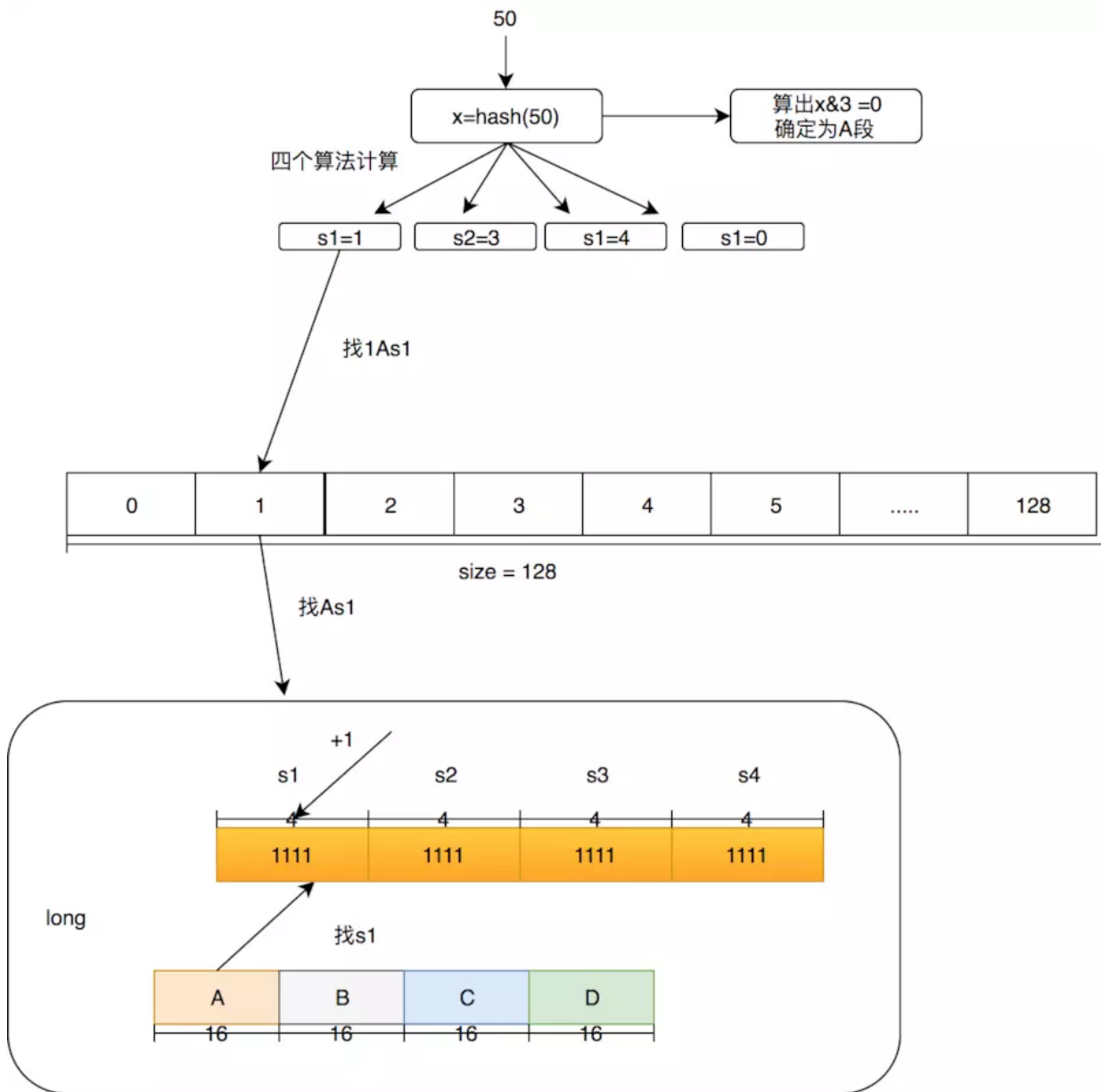
在Count-Min Sketch中，我这里直接说caffeine中的实现吧(在FrequencySketch这个类中),如果你的缓存大小是100，他会生成一个long数组大小是和100最接近的2的幂的数，也就是128。而这个数组将会记录我们的访问频率。在caffeine中规定频率最大为15，15的二进制位1111，总共是4位，而Long型是64位。所以每个Long型可以放16种算法，但是caffeine并没有这么做，只用了四种hash算法，每个Long型被分为四段，每段里面保存的是四个算法的频率。这样做的好处是可以进一步减少Hash冲突，原先128大小的hash，就变成了128X4。

一个Long的结构如下:



我们的4个段分为A,B,C,D，在后面我也会这么叫它们。而每个段里面的四个算法我叫他s1,s2,s3,s4。下面举个例子如果要添加一个访问50的数字频率应该怎么做？我们这里用size=100来举例。

1. 首先确定50这个hash是在哪个段里面，通过 $\text{hash} \& 3$ (3的二进制是11)必定能获得小于4的数字，假设 $\text{hash} \& 3 = 0$ ，那就在A段。
2. 对50的hash再用其他hash算法再做一次hash，得到long数组的位置，也就是在长度128数组中的位置。假设用s1算法得到1，s2算法得到3，s3算法得到4，s4算法得到0。
3. 因为S1算法得到的是1，所以在long[1]的A段里面的s1位置进行+1,简称1As1加1，然后在3As2加1，在4As3加1，在0As4加1。

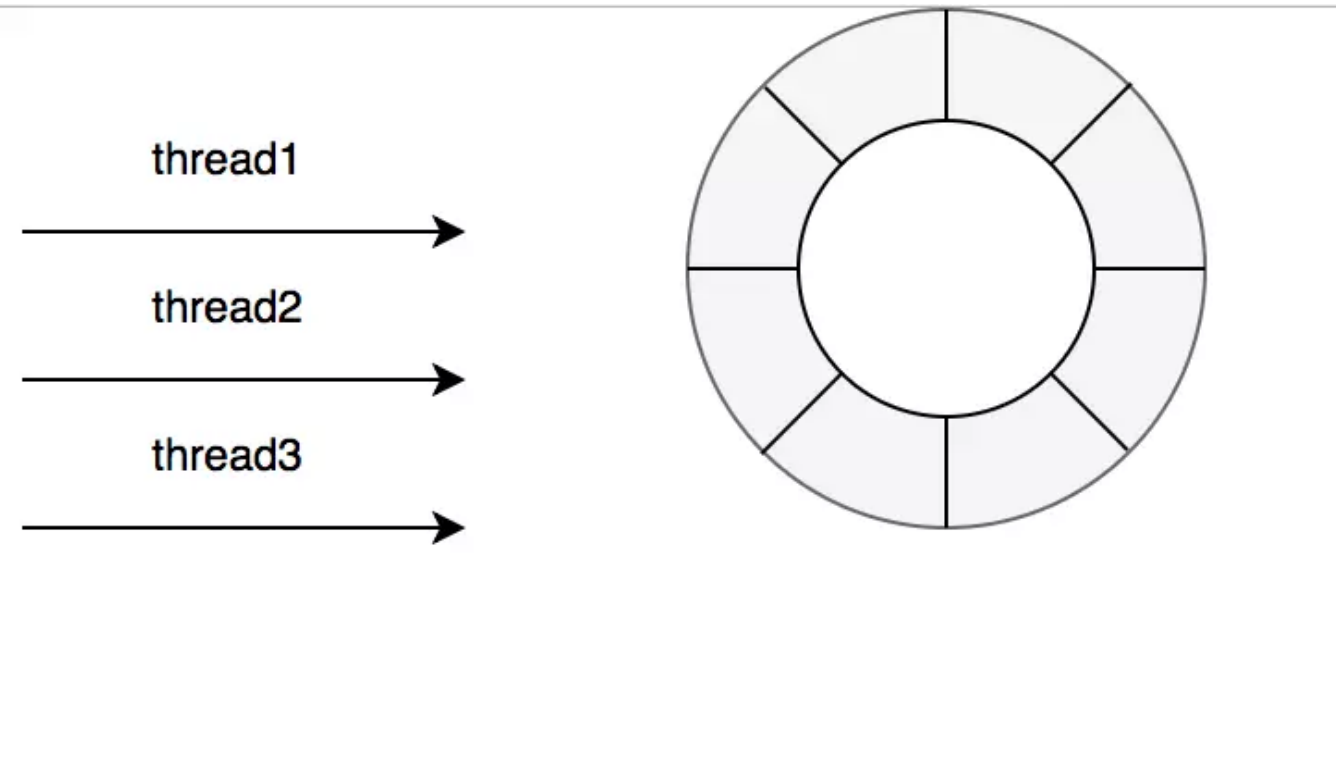


这个时候有人会质疑频率最大为15的这个是否太小？没关系在这个算法中，比如size等于100，如果他全局提升了 $\text{size} \times 10$ 也就是1000次就会全局除以2衰减，衰减之后也可以继续增加，这个算法再W-TinyLFU的论文中证明了其可以较好的适应时间段的访问频率。

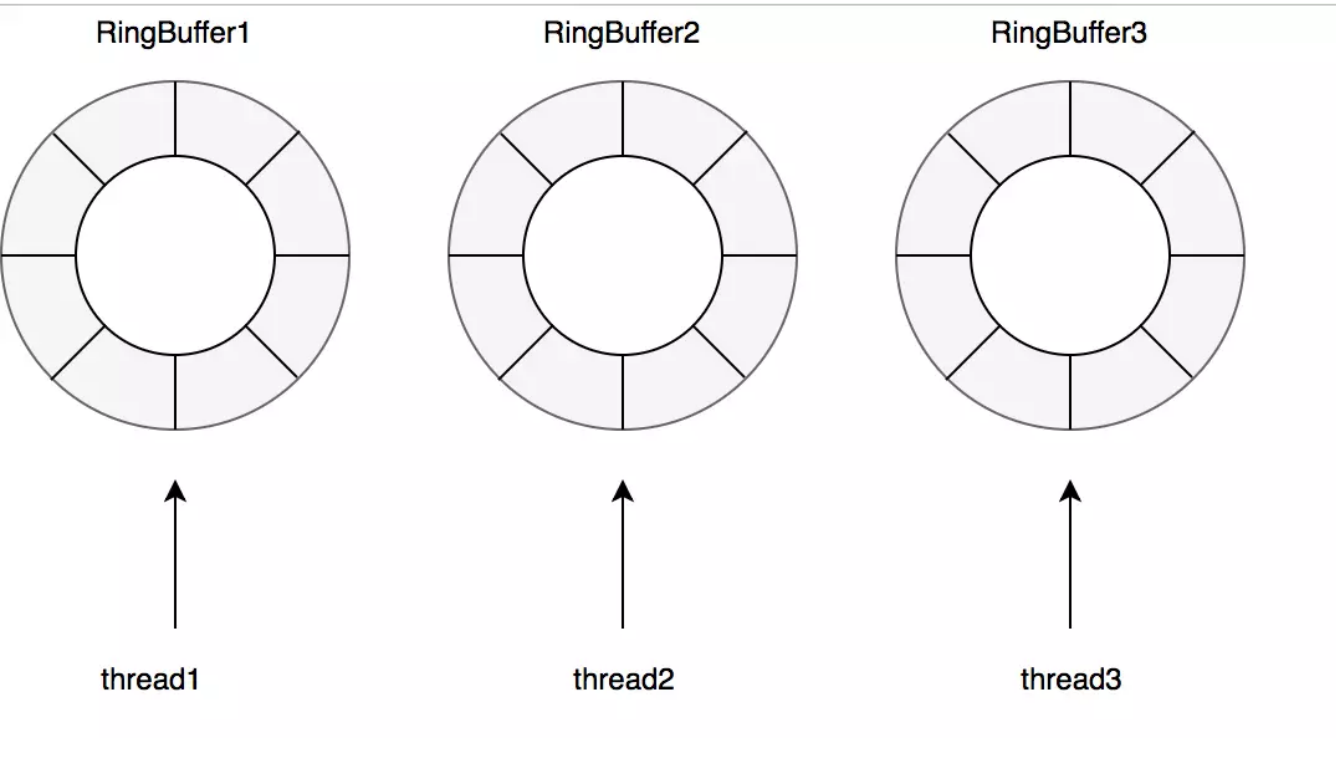
2.2读写性能

在guava cache中我们说过其读写操作中夹杂着过期时间的处理，也就是你在一次Put操作中有可能还会做淘汰操作，所以其读写性能会受到一定影响，可以看上面的图中，caffeine的确在读写操作上面完爆guava cache。主要是在caffeine，对这些事件的操作是通过异步操作，他将事件提交至队列，这里的队列的数据结构是RingBuffer，不清楚的可以看看这篇文章[你应该知道的高性能无锁队列Disruptor](#)。然后通过默认的ForkJoinPool.commonPool()，或者自己配置线程池，进行取队。
作 然后在进行后续的淘汰 过期操作

当然读写也是有不同的队列，在caffeine中认为缓存读比写多很多，所以对于写操作是所有线程共享一个Ringbuffer。



对于读操作比写操作更加频繁，进一步减少竞争，其为每个线程配备了一个RingBuffer：

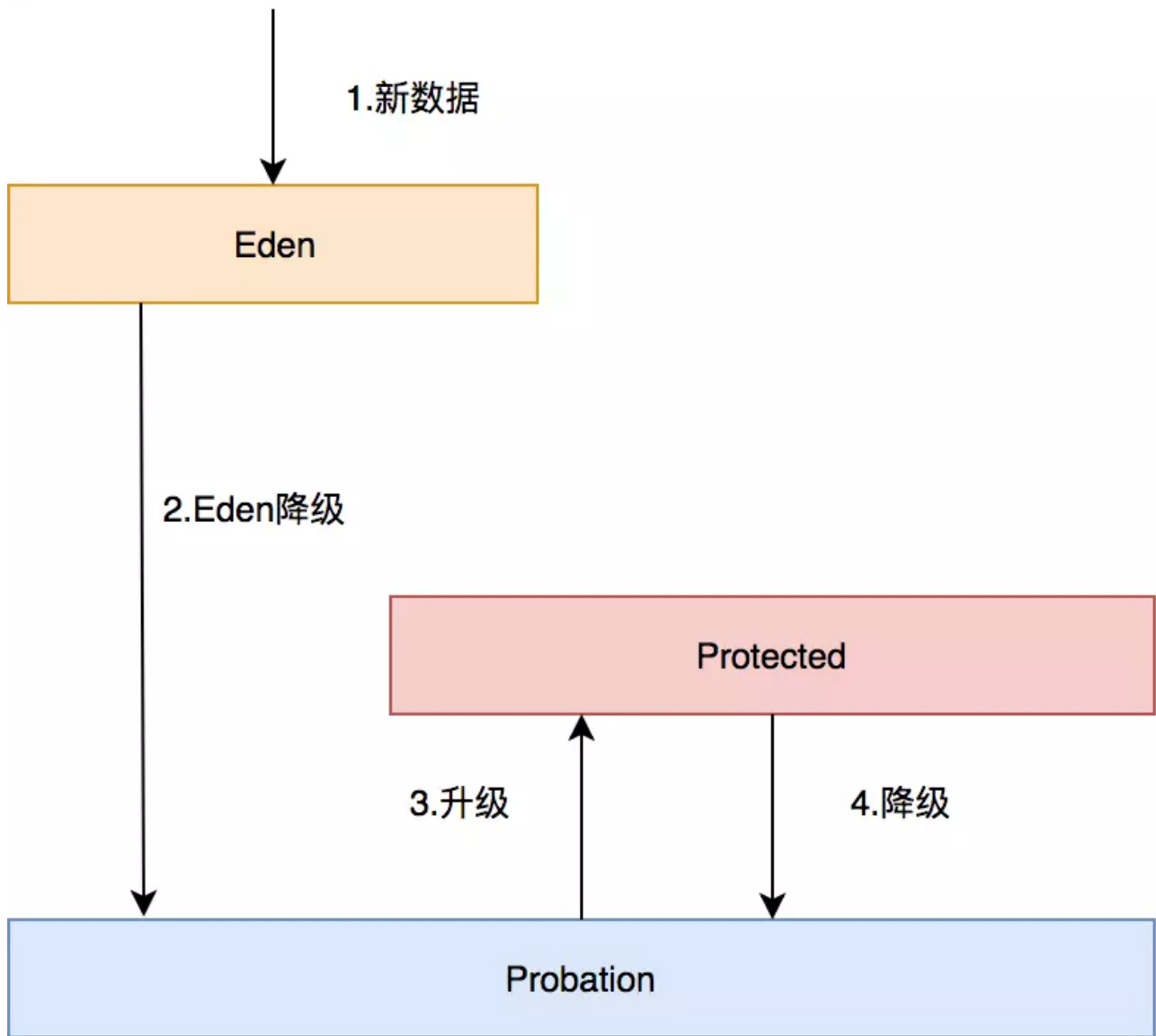


2.3数据淘汰策略

在caffeine所有的数据都在ConcurrentHashMap中，这个和guava cache不同，guava cache是自己实现了个类似ConcurrentHashMap的结构。在caffeine中有三个记录引用的LRU队列：

- Eden队列:在caffeine中规定只能为缓存容量的%1,如果size=100,那这个队列的有效大小就等于1。这个队列中记录的是新到的数据，防止突发流量由于之前没有访问频率，而导致被淘汰。比如有一部新剧上线，在最开始其实是没有访问频率的，防止上线之后被其他缓存淘汰出去，而加入这个区域。伊甸区，最舒服最安逸的区域，在这里很难被其他数据淘汰。
- Probation队列:叫做缓刑队列，在这个队列就代表你的数据相对比较冷，马上就要被淘汰了。这个有效大小为size减去eden减去protected。
- Protected队列:在这个队列中，可以稍微放心一下了，你暂时不会被淘汰，但是别急，如果Probation队列没有数据了或者Protected数据满了，你也将会被面临淘汰的尴尬局面。当然想要变成这个队列，需要把Probation访问一次之后，就会提升为Protected队列。这个有效大小为(size减去eden) X 80% 如果size =100，就会是79。

这三个队列关系如下：



1. 所有的新数据都会进入Eden。
2. Eden满了，淘汰进入Probation。
3. 如果在Probation中访问了其中某个数据，则这个数据升级为Protected。
4. 如果Protected满了又会继续降级为Probation。

对于发生数据淘汰的时候，会从Probation中进行淘汰。会把这个队列中的数据队头称为受害者，这个队头肯定是最早进入的，按照LRU队列的算法的话那他其实他就应该被淘汰，但是在这里只能叫他受害者，这个队列是缓刑队列，代表马上要给他行刑了。这里会取出队尾叫候选者，也叫攻击者。这里受害者会和攻击者皇城PK决出我们应该被淘汰的。



通过我们的Count-Min Sketch中的记录的频率数据有以下几个判断:

- 如果攻击者大于受害者，那么受害者就直接被淘汰。
- 如果攻击者 ≤ 5 ，那么直接淘汰攻击者。这个逻辑在他的注释中有解释:

```
int candidateFreq = frequencySketch().frequency(candidateKey);
if (candidateFreq > victimFreq) {
    return true;
} else if (candidateFreq <= 5) {
    // The maximum frequency is 15 and halved to 7 after a reset to age the history. An attack
    // exploits that a hot candidate is rejected in favor of a hot victim. The threshold of a warm
    // candidate reduces the number of random acceptances to minimize the impact on the hit rate.
    return false;
}
int random = ThreadLocalRandom.current().nextInt();
return ((random & 127) == 0);
```

他认为设置一个预热的门槛会让整体命中率更高。

- 其他情况，随机淘汰。

3.Caffeine功能剖析

在Caffeine中功能比较多，下面来剖析一下，这些API到底是如何生效的呢？

3.1 百花齐放-Cache工厂

在Caffeine中有个LocalCacheFactory类，他会根据你的配置进行具体Cache的创建。

```
static <K, V> BoundedLocalCache<K, V> newBoundedLocalCache(Caffeine<K, V> builder,
    CacheLoader<? super K, V> cacheLoader, boolean async) {
    StringBuilder sb = new StringBuilder();
    if (builder.isStrongKeys()) {
        sb.append('S');
    } else {
        sb.append('W');
    }
    if (builder.isStrongValues()) {
        sb.append('S');
    } else {
        sb.append('I');
    }
    if (builder.removalListener != null) {
        sb.append("Li");
    }
    if (builder.isRecordingStats()) {
        sb.append('S');
    }
    if (builder.evicts()) {
        sb.append('M');
        if (builder.isWeighted()) {
            sb.append('W');
        } else {
            sb.append('S');
        }
    }
    if (builder.expiresAfterAccess() || builder.expiresVariable()) {
        sb.append('A');
    }
    if (builder.expiresAfterWrite()) {
        sb.append('W');
    }
    if (builder.refreshes()) {
        sb.append('R');
    }
}
```

可以看见他会根据你是否配置了过期时间,remove监听器等参数,来进行字符串的拼装,最后会根据字符串来生成具体的Cache,这里的Cache太多了,作者的源码并没有直接写这部分代码,而是通过Java Poet进行代码的生成:


```

switch (sb.toString()) {
    case "SI": return new SI<>(builder, cacheLoader, async);
    case "SIA": return new SIA<>(builder, cacheLoader, async);
    case "SIAR": return new SIAR<>(builder, cacheLoader, async);
    case "SIAW": return new SIAW<>(builder, cacheLoader, async);
    case "SIAWR": return new SIAWR<>(builder, cacheLoader, async);
    case "SILi": return new SILi<>(builder, cacheLoader, async);
    case "SILiA": return new SILiA<>(builder, cacheLoader, async);
    case "SILiAR": return new SILiAR<>(builder, cacheLoader, async);
    case "SILiAW": return new SILiAW<>(builder, cacheLoader, async);
    case "SILiAWR": return new SILiAWR<>(builder, cacheLoader, async);
    case "SILiMS": return new SILiMS<>(builder, cacheLoader, async);
    case "SILiMSA": return new SILiMSA<>(builder, cacheLoader, async);
    case "SILiMSAR": return new SILiMSAR<>(builder, cacheLoader, async);
    case "SILiMSAW": return new SILiMSAW<>(builder, cacheLoader, async);
    case "SILiMSAWR": return new SILiMSAWR<>(builder, cacheLoader, async);
    case "SILiMSR": return new SILiMSR<>(builder, cacheLoader, async);
    case "SILiMSW": return new SILiMSW<>(builder, cacheLoader, async);
    case "SILiMSWR": return new SILiMSWR<>(builder, cacheLoader, async);
    case "SILiMW": return new SILiMW<>(builder, cacheLoader, async);
    case "SILiMWA": return new SILiMWA<>(builder, cacheLoader, async);
    case "SILiMWAR": return new SILiMWAR<>(builder, cacheLoader, async);
    case "SILiMWAW": return new SILiMWAW<>(builder, cacheLoader, async);
    case "SILiMWAWR": return new SILiMWAWR<>(builder, cacheLoader, async);
    case "SILiMWR": return new SILiMWR<>(builder, cacheLoader, async);
    case "SILiMW": return new SILiMW<>(builder, cacheLoader, async);
    case "SILiMWR": return new SILiMWR<>(builder, cacheLoader, async);
    case "SILiR": return new SILiR<>(builder, cacheLoader, async);
    case "SILiS": return new SILiS<>(builder, cacheLoader, async);
    case "SILiSA": return new SILiSA<>(builder, cacheLoader, async);
    case "SILiSAR": return new SILiSAR<>(builder, cacheLoader, async);
    case "SILiSAW": return new SILiSAW<>(builder, cacheLoader, async);
    case "SILiSAWR": return new SILiSAWR<>(builder, cacheLoader, async);
    case "SILiSMS": return new SILiSMS<>(builder, cacheLoader, async);
    case "SILiSMSA": return new SILiSMSA<>(builder, cacheLoader, async);
    case "SILiSMSAR": return new SILiSMSAR<>(builder, cacheLoader, async);
    case "SILiSMSAW": return new SILiSMSAW<>(builder, cacheLoader, async);
    case "SILiSMSAWR": return new SILiSMSAWR<>(builder, cacheLoader, async);
}

```

3.2 转瞬即逝-过期策略

在Caffeine中分为两种缓存，一个是有界缓存，一个是无界缓存，无界缓存不需要过期并且没有界限。在有界缓存中提供了三个过期API:

- `expireAfterWrite` : 代表着写了之后多久过期。
- `expireAfterAccess`: 代表着最后一次访问了之后多久过期。
- `expireAfter`: 在`expireAfter`中需要自己实现`Expiry`接口，这个接口支持`create`, `update`, 以及`access`了之后多久过期。注意这个API和前面两个API是互斥的。这里和前面两个API不同的是，需要告诉缓存框架他应该在具体的某个时间过期，也就是通过前面的重写`create`, `update`以及

在Caffeine中有个scheduleDrainBuffers方法,用来进行我们的过期任务的调度,在我们读写之后都会对其进行调用:

```
/**
 * Attempts to schedule an asynchronous task to apply the pending operations to the page
 * replacement policy. If the executor rejects the task then it is run directly.
 */
void scheduleDrainBuffers() {
    if (drainStatus() >= PROCESSING_TO_IDLE) {
        return;
    }
    if (evictionLock.tryLock()) {
        try {
            int drainStatus = drainStatus();
            if (drainStatus >= PROCESSING_TO_IDLE) {
                return;
            }
            lazySetDrainStatus(PROCESSING_TO_IDLE);
            executor().execute(drainBuffersTask);
        } catch (Throwable t) {
            logger.log(Level.WARNING, msg: "Exception thrown when submitting maintenance task", t);
            maintenance(/* ignored */ task: null);
        } finally {
            evictionLock.unlock();
        }
    }
}
```

首先他会进行加锁,如果锁失败说明有人已经在执行调度了。他会使用默认的线程池ForkJoinPool或者自定义线程池,这里的drainBuffersTask其实是Caffeine中PerformCleanupTask。

```
/** A reusable task that performs the maintenance work; used to avoid wrapping by ForkJoinPool. */
final class PerformCleanupTask extends ForkJoinTask<Void> implements Runnable {
```

```
void performCleanUp(@Nullable Runnable task) {
    evictionLock.lock();
    try {
        maintenance(task);
    } finally {
        evictionLock.unlock();
    }
}
```

在performCleanUp方法中再次进行加锁,防止其他线程进行清理操作。然后我们进入到maintenance方法中:


```
void maintenance(@Nullable Runnable task) {
    lazySetDrainStatus(PROCESSING_TO_IDLE);

    try {
        drainReadBuffer();

        drainWriteBuffer();
        if (task != null) {
            task.run();
        }

        drainKeyReferences();
        drainValueReferences();

        expireEntries();
        evictEntries();
    } finally {
        if ((drainStatus() != PROCESSING_TO_IDLE) || !casDrainStatus(PROCESSING_TO_IDLE, IDLE)) {
            lazySetDrainStatus(REQUIRED);
        }
    }
}
```

可以看见里面有挺多方法的，其他方法稍后再讨论，这里我们重点关注`expireEntries()`，也就是用来过期的方法：

```
void expireEntries() {
    long now = expirationTicker().read();
    expireAfterAccessEntries(now);
    expireAfterWriteEntries(now);
    expireVariableEntries(now);
}
```

- 首先获取当前时间。
- 第二步,进行`expireAfterAccess`的过期:

```
void expireAfterAccessEntries(long now) {
    if (!expiresAfterAccess()) {
        return;
    }

    expireAfterAccessEntries(accessOrderEdenDeque(), now);
    if (evicts()) {
        expireAfterAccessEntries(accessOrderProbationDeque(), now);
        expireAfterAccessEntries(accessOrderProtectedDeque(), now);
    }
}
```

```
void expireAfterAccessEntries(AccessOrderDeque<Node<K, V>> accessOrderDeque, long now) {
    long duration = expiresAfterAccessNanos();
    for (;;) {
        Node<K, V> node = accessOrderDeque.peekFirst();
        if ((node == null) || ((now - node.getAccessTime()) < duration)) {
            return;
        }
        evictEntry(node, RemovalCause.EXPIRED, now);
    }
}
```

这里根据我们的配置evicts()方法为true，所以会从三个队列都进行过期淘汰，上面已经说过了这三个队列都是LRU队列，所以我们的expireAfterAccessEntries方法，只需要把各个队列的头结点进行判断是否访问过期然后进行剔除即可。

- 第三步,是expireAfterWrite:

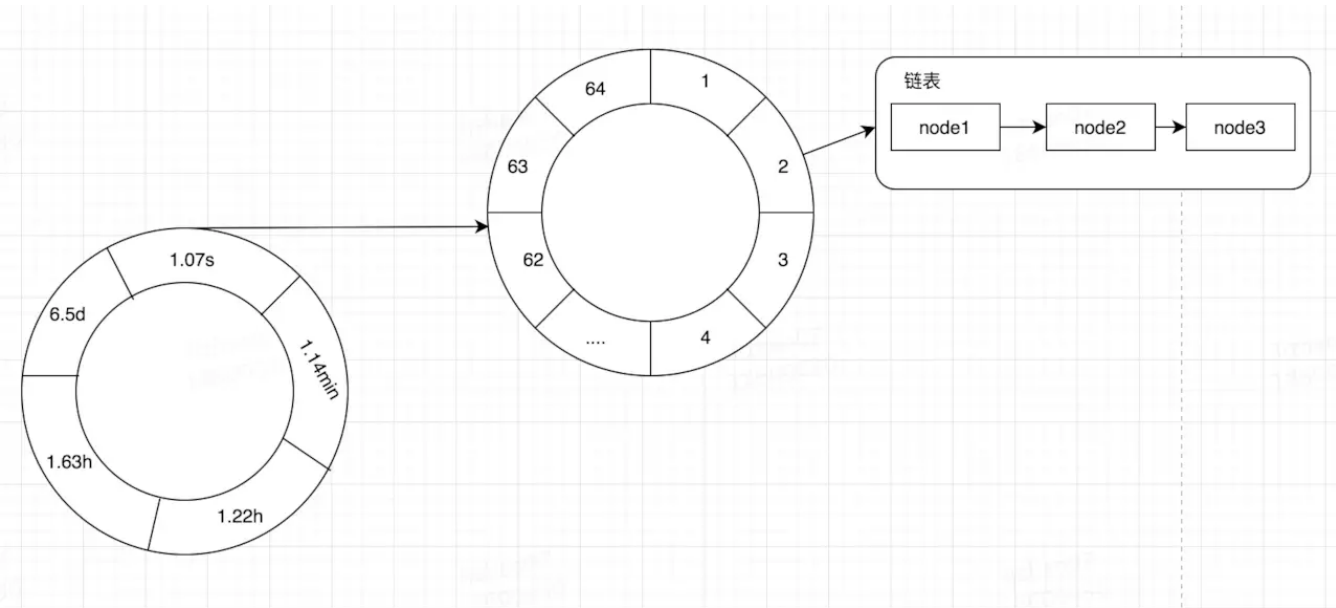
```
void expireAfterWriteEntries(long now) {
    if (!expiresAfterWrite()) {
        return;
    }
    long duration = expiresAfterWriteNanos();
    for (;;) {
        final Node<K, V> node = writeOrderDeque().peekFirst();
        if ((node == null) || ((now - node.getWriteTime()) < duration)) {
            break;
        }
        evictEntry(node, RemovalCause.EXPIRED, now);
    }
}
```

可以看见这里依赖了一个队列writeQrderDeque,这个队列的数据是什么时候填充的呢？当然也是使用异步，具体方法在我们上面的drainWriteBuffer中，他会将我们之前放进RingBuffer的Task拿出来执行，其中也包括添加writeQrderDeque。过期的策略很简单，直接循环弹出第一个判断其是否过期即可。

- 第四步，进行expireVariableEntries过期:

```
void expireVariableEntries(long now) {
    if (expiresVariable()) {
        timerWheel().advance(now);
    }
}
```


数组。在Caffeine中是一个二层时间轮，也就是二维数组，其一维的数据表示较大的时间维度比如，秒，分，时，天等，其二维的数据表示该时间维度较小的时间维度，比如秒内的某个区间段。当定位到一个TimeWhile[i][j]之后，其数据结构其实是一个链表，记录着我们的Node。在Caffeine利用时间轮记录我们在某个时间过期的数据，然后去处理。



在Caffeine中的时间轮如上面所示。在我们插入数据的时候，根据我们重写的方法计算出他应该过期的时间，比如他应该在1536046571142时间过期，上一次处理过期时间是1536046571100，对其相减则得到42ms，然后将其放入时间轮，由于其小于1.07s，所以直接放入1.07s的位置，以及第二层的某个位置(需要经过一定的算法算出)，使用尾插法插入链表。

处理过期时间的时候会算出上一次处理的时间和当前处理的时间的差值，需要将其这个时间范围之内的所有时间轮的时间都进行处理，如果某个Node其实没有过期，那么就需要将其重新插入进时间轮。

3.3.除旧布新-更新策略

Caffeine提供了refreshAfterWrite()方法来让我们进行写后多久更新策略:

```
Cache<String, String> cache = Caffeine.newBuilder()
    .refreshAfterWrite(1,TimeUnit.SECONDS)
    .build(new CacheLoader<String, String>() {
        @Override
        public String load(String key) throws Exception {
            return "key"+"我是刷新的key";
        }
    })
```

上面的代码我们需要建立一个CacheLodaer来进行刷新,这里是同步进行的,可以通过buildAsync方法进行异步构建。在实际业务中这里可以把我们代码中的mapper传入进去,进行数据源的刷新。

注意这里的刷新并不是到期就刷新,而是对这个数据再次访问之后,才会刷新。举个例子:有个key:'咖啡',value:'拿铁'的数据,我们设置1s刷新,我们在添加数据之后,等待1分钟,按理说下次访问时他会刷新,获取新的值,可惜并没有,访问的时候还是返回'拿铁'。但是继续访问的话就会发现,他已经进行了刷新了。

我们来看看自动刷新他是怎么做的呢?自动刷新只存在读操作之后,也就是我们afterRead()这个方法,其中有个方法叫refreshIfNeeded,他会根据你是同步还是异步然后进行刷新处理。

3.4 虚虚实实-软引用和弱引用

在Java中有四种引用类型:强引用 (StrongReference)、软引用 (SoftReference)、弱引用 (WeakReference)、虚引用 (PhantomReference)。

- 强引用:在我们代码中直接声明一个对象就是强引用。
- 软引用:如果一个对象只具有软引用,如果内存空间足够,垃圾回收器就不会回收它;如果内存空间不足了,就会回收这些对象的内存。只要垃圾回收器没有回收它,该对象就可以被程序使用。软引用可以和一个引用队列 (ReferenceQueue) 联合使用,如果软引用所引用的对象被垃圾回收器回收,Java虚拟机就会把这个软引用加入到与之关联的引用队列中。
- 弱引用:在垃圾回收器线程扫描它所管辖的内存区域的过程中,一旦发现了只具有弱引用的对象,不管当前内存空间足够与否,都会回收它的内存。弱引用可以和一个引用队列 (ReferenceQueue) 联合使用,如果弱引用所引用的对象被垃圾回收,Java虚拟机就会把这个弱引用加入到与之关联的引用队列中。
- 虚引用:如果一个对象仅持有虚引用,那么它就和没有任何引用一样,在任何时候都可能被垃圾回收器回收。虚引用必须和引用队列 (ReferenceQueue) 联合使用。当垃圾回收器准备回收一个对象时,如果发现它还有虚引用,就会在回收对象的内存之前,把这个虚引用加入到与之 关联的引用队列中。

3.4.1弱引用的淘汰策略

在Caffeine中支持弱引用的淘汰策略,其中有两个api: weakKeys()和weakValues(),用来设置key是弱引用还是value是弱引用。具体原理是在put的时候将key和value用虚引用进行包装并绑定至引用队列:

。

具体回收的时候，在我们前面介绍的maintenance方法中，有两个方法：

```
//处理key引用的  
drainKeyReferences();  
//处理value引用  
drainValueReferences();
```

具体的处理的代码有：

因为我们的key已经被回收了，然后他会进入引用队列，通过这个引用队列，一直弹出到他为空为止。我们能根据这个队列中的运用获取到Node,然后对其进行驱逐。

注意:很多同学以为在缓存中内部是存储的Key-Value的形式，其实存储的是KeyReference - Node(Node中包含Value)的形式。

3.4.2 软引用的淘汰策略

在Caffeine中还支持软引用的淘汰策略,其api是softValues(),软引用只支持Value不支持Key。我们可以看见在Value的回收策略中有：

和key引用回收相似，但是要说明的是这里的引用队列，有可能是软引用队列，也有可能是弱引用队列。

3.5知己知彼-打点监控

在Caffeine中提供了一些的打点监控策略，通过recordStats()Api进行开启，默认是使用Caffeine自带的，也可以自己进行实现。在StatsCounter接口中，定义了需要打点的方法目前来说有如下几个：

- recordHits：记录缓存命中
- recordMisses：记录缓存未命中
- recordLoadSuccess：记录加载成功(指的是CacheLoader加载成功)
- recordLoadFailure：记录加载失败
- recordEviction：记录淘汰数据

通过上面的监听，我们可以实时监控缓存当前的状态，以评估缓存的健康程度以及缓存命中率等，方便后续调整参数。

3.6有始有终-淘汰监听

有很多时候我们需要知道Caffeine中的缓存为什么被淘汰了呢，从而进行一些优化？这个时候我们就需要一个监听器,代码如下所示：

```
Cache<String, String> cache = Caffeine.newBuilder()
```

```
    })))  
    .build();
```

在Caffeine中被淘汰的原因有很多种:

- EXPLICIT: 这个原因是, 用户造成的, 通过调用remove方法从而进行删除。
- REPLACED: 更新的时候, 其实相当于把老的value给删了。
- COLLECTED: 用于我们的垃圾收集器, 也就是我们上面减少的软引用, 弱引用。
- EXPIRED: 过期淘汰。
- SIZE: 大小淘汰, 当超过最大的时候就会进行淘汰。

当我们进行淘汰的时候就会进行回调, 我们可以打印出日志, 对数据淘汰进行实时监控。

4.最后

本文介绍了Caffeine的全部功能原理, 其中的知识点涉及到:LFU,LRU,时间轮, Java的四种引用等等。如果你对Caffeine不感兴趣也没有关系, 通过这些知识的介绍相信你也收获了不少。最后关于缓存系列基本也告一段落, 如果还想了解更多可以关注我的公众号, 加我好友或者加入技术交流微信群进行讨论。

最后这篇文章被我收录于JGrowing, 一个全面, 优秀, 由社区一起共建的Java学习路线, 如果您想参与开源项目的维护, 可以一起共建, github地址为:[github.com/javagrowing...](https://github.com/javagrowing) 麻烦给个小星星哟。

如果你觉得这篇文章对你有帮助, 可以关注我的技术公众号, 最近作者收集了很多最新的学习资料视频以及面试资料, 关注之后即可领取, 你的关注和转发是对我最大的支持, O(∩_∩)O

关注下面的标签，发现更多相似文章

- 后端
- Java
- 算法

咖啡拿铁 后端开发 公众号:【咖啡拿铁】 @ 猿辅导
获得点赞 4,733 次 · 文章被阅读 119,421 次

关注

安装掘金浏览器插件

打开新标签页发现好内容，掘金、GitHub、Dribbble、ProductHunt 等站点内容轻松获取。快来安装掘金浏览器插件获取高质量内容吧！

评论

输入评论...

Era

作者，关于你讲道德频率记录，我个人认为在每次使用get获取数据的时候，就会进行一次频率算法记录，但是我本地在FrequencySketch这个类里打了断点，但是debug并没有进到这个类。所以说，频率记录发生在什么时候？

21天前

👍

🗨 回复

👤 注册

登录

回复 Era: 将到的频率记录，（手误，打错了）

21天前

咖啡拿铁 (作者) 后端开发 公众号:...

回复 Era: 没有吗 😊 太久远了不是很清楚了

21天前

ben-manes

You might enjoy reading our latest work to make W-TinyLfu adaptive. I am still implementing it (v2.7) but we have a paper and simulations. We're using hill climbing to dynamically tune the recency/frequency configuration. Thanks for these articles, very well done.

1月前



回复

myshzzx

caffeine 读写并不是异步的...

4月前



回复

咖啡拿铁 (作者) 后端开发 公众号:...

回复 myshzzx: 读写是同步的 但是淘汰操作这些是异步的

3月前

相关推荐

专栏 · 陈续渊 · 1天前 · 后端 / Spring Cloud

[Spring Cloud Tutorial翻译系列]微服务-定义、原则、好处

2

5

专栏 · 埃里克拓荒 · 14小时前 · 算法

五线谱入门，程序员也可以玩音乐

3

1

专栏 · 吾乃上将军邢道荣 · 14小时前 · 后端

SpringBoot踩坑日记-一个非空校验引发的bug

2

专栏 · lijiantao · 17小时前 · Java

Java并发 之 线程组 ThreadGroup 介绍

2

尴尬的面试现场：说说你们系统有多大QPS？系统到底怎么抗住高并发的？【石杉的架构笔记】

 148

 37

专栏 · Sophie May · 16小时前 · Java

Java多线程之Callable,Future,FutureTask

 1



专栏 · 架构师修行之路 · 18小时前 · 算法

程序猿修仙之路--算法之选择排序

 1



专栏 · 金空空 · 19小时前 · Java

Java填坑系列之LinkedList





专栏 · 掘金翻译计划 · 16小时前 · 后端 / 掘金翻译计划

[译] 多线程简介：一步一步来接近多线程的世界

 19

 1

专栏 · Java劝退师 · 2小时前 · Java

Dubbo源码解析之服务导出过程





